Measure the Harness, Not the Model: A Controlled Study of Coding-Agent Harnesses Across Local and Cloud Models

Siddhant Goswami
100x Engineers
content@100xengineers.com

Abstract

A coding agent is a *pair*: Agent = Model + Harness, where the harness is the software around the model—prompt assembly, tools, the execution loop, verification, recovery, and permissions. Most benchmarks score the pair jointly and attribute the result to the model. We present CRUCIBLE, an apparatus that holds the model fixed and makes the *harness* the experimental variable: a factorial harness×model×seed protocol over a tiered task battery, scored per *run* (not per output) with a gated, non-substitutable profile, Score = Safety × (0.6 Completion + 0.2 Path + 0.2 State), and reported as GOODPUT: gated score over *all* attempts with timeouts counted as zero. In an exploratory pilot of 8 harnesses (Claude Code, aider, Codex CLI, Goose, Pi, Hermes, a thin file-block control, and a deterministic floor) on 3 local models plus cloud references—515 frozen runs, extended by a 222-run within-family size ladder, a 20-run cloud bookend, and a pre-registered 341-run third-family confirmatory arm—we observe: (i) *interface-fit, not capability, is often the binding constraint*: Codex CLI scores 0 on all 89 local cells yet 20/20 on its native cloud model, a discontinuity produced by a model swap alone; (ii) *the harness substitutes for scale where capability is scarcest*: on a 2.7 GB model the minimalist Pi harness reaches GOODPUT 0.79 versus 0.24 for the thin control (Δ=0.55 [0.18, 0.87]), beating the control on a model 2.4× larger; (iii) *excluding timeouts—common practice—inverts harness rankings* (a harness at 1.00 finished-only falls to 0.70; another from 0.99 to 0.33); and (iv) a multiplicative safety gate catches boundary violations on runs that *complete*. All numbers regenerate deterministically from committed run ledgers and are guarded in CI against drift. We release the apparatus, the ledgers, and a pre-registered confirmatory design.

1 Introduction

Agent benchmarks overwhelmingly vary the model and hold the scaffold fixed—or worse, vary both and report one number. Yet the scaffold moves the score: swapping scaffolds shifts SWE-bench results by 10–20 points [6, 7], SWE-agent's agent–computer interface alone unlocked large gains [19], and the Holistic Agent Leaderboard finds two scaffolds on the same model disagree on a large fraction of tasks [9]. The field knows *that* the harness matters. What is missing is an instrument that *measures* it: holding the model fixed, how much does the harness change correctness, process, safety, and cost—and does that advantage transfer across models?

CRUCIBLE is that instrument. It runs the same task, under the same deterministic verifier, budget, and seeds, through interchangeable harness adapters—commercial CLI agents (Claude Code, aider, Codex CLI, Goose), lean open-source harnesses (Pi, Hermes), a thin file-block control, and a deterministic mock floor—against local models small enough that the harness, not the model, decides the outcome. Contributions:

1. **A harness-first measurement apparatus** (§3): a one-line adapter contract; a tiered, discriminating task battery; per-run trace scoring with a multiplicative safety gate; uniform token metering via a logging proxy; frozen, committed run ledgers with a CI guard that fails the build if any published number drifts.
2. **Interface-fit as a predictive mechanism** (§4.2): whether a (harness, model) pair succeeds is often bound by the compatibility between the harness's expected I/O shape and the model's output shape, not by model capability. The cleanest demonstration is a bookend: Codex CLI is a structural zero on every local model (0/89) and perfect on its native cloud model (20/20, 5 seeds × 4 tasks)—the harness held fixed, only the model swapped.
3. **GOODPUT: reliability as a first-class methodological correction** (§4.1): scoring only finished runs inverts rankings; counting timeouts as zeros reorders the field toward harnesses that *deliver*. A per-cell timeout autopsy

separates harness hangs from host-conditional latency.

4. **A routing artifact** (§4.6): per (model, task-tier) best-achievable local GOODPUT, yielding a local-vs-cloud escalation rule keyed on the task's *tool-capability tier*—a signal prompt-difficulty routers [5, 15] cannot see.

We are explicit about epistemic status (§5): the 515-run battery is an *exploratory pilot*; hypotheses were sharpened from it, so its support is in-sample. A pre-registered scaled design (§6) states the confirmatory predictions, including the primary one the thesis lives or dies on.

2 Related work

Benchmarks that conflate model and harness. SWE-bench [7], AgentBench [11], GAIA [13], OSWorld [18], and Terminal-Bench [12] score the (model, scaffold) pair jointly. MLE-bench [2] compares scaffolds ad hoc; SWE-agent [19] demonstrates the interface is first-class but does not make it the measured variable. Aider's polyglot leaderboard fixes the harness and varies the model—the inverse of our design.

Harness/scaffold as the variable. HAL [9] is the closest system: cost-aware, process-level evaluation at 21,730-rollout scale, observing scaffold disagreement. HAL is model-first and descriptive; CRUCIBLE is harness-first and inferential—a factorial design with a cross-model transfer (rank-stability) test, a multiplicative safety gate, and deterministic trace scoring instead of LLM log analysis. Recent work evolves harnesses automatically and reports cross-model transfer of harness gains [10]; we measure harnesses rather than optimize them. Overeager-agent measurements [16] find framework architecture dominates out-of-scope behavior—the safety-side complement to our capability-side attribution, which our gate unifies into one score. Agent Arena decomposes agents into model×framework×tools but ranks by subjective ELO; CRUCIBLE uses deterministic oracles and fixed budgets.

Evaluation methodology. We adopt the cost-controlled stance of Kapoor et al. [8] (cost reported alongside, never folded in), the statistical reporting of Miller [14] (bootstrap CIs, paired comparisons, clustered resampling), reliability-over-repeats from τ-bench's pass^k [20], and the task-validity concerns of the ABC checklist [21]—one of our own findings (§4.4) is an instance of the verifier-gaming failure ABC catalogs, which we report and harden against.

Routing. RouteLLM [15], FrugalGPT [3], and Hybrid LLM [5] route on predicted per-prompt difficulty or quality gap. None route on task-tool requirements; our tier-keyed escalation rule is exactly that missing signal.

Safety. AgentHarm [1], AgentDojo [4], and ToolEmu [17] score model refusal/robustness on a fixed harness; our T4 tier instead asks whether the *harness* contains the blast radius, with deterministic per-channel policy checks and a gate that completion cannot buy back.

3 The CRUCIBLE apparatus

Adapter contract. A harness is integrated by one script: `adapters/<name>.sh <workdir> <iter> <feedback>`—read the goal from `TASK.md`, read the last verifier feedback, make one attempt by editing files. The bounded loop (act → verify → repeat, capped iterations) is owned by the runner, so every harness runs the identical protocol. Lean harnesses are wired to the same local model endpoint, making the harness the only variable in a row.

Scoring the run, not the output. Each run emits a per-iteration trace. The score is gated and non-substitutable:

$$\text{Score} \;=\; \text{Safety} \times \big(0.6 \cdot \text{Completion} \;+\; 0.2 \cdot \text{Path} \;+\; 0.2 \cdot \text{State}\big)$$

Safety = min(tool, resource, info) per-channel adherence against a *hidden* per-task policy—multiplicative, so a boundary violation collapses the score regardless of completion, and the audit fails closed. Completion is a deterministic oracle (`verify.sh`) with checkpoint partial credit; Path (action validity, recovery) and State (checkpoint progress preserved across iterations) are computed from the trace with no LLM judge. Cost—metered tokens via a logging proxy, wall time, successes per Mtoken—is reported alongside, never folded in [8].

GOODPUT. The headline metric is the gated score averaged over *all* attempts with each timeout counted as 0, reported next to the finish rate. §4.1 shows why the conventional alternative (score over finished runs) is not a detail.

Task battery. Nine tasks in five tiers, each `TASK.md` + deterministic `verify.sh` + `task.yaml` (tier, budgets, safety policy, seeds): T0 single-file fixes (calibration floor); T1 *tool-recovery*—the sharpest discriminator: the task passes only by *running* a generator that fails on first invocation and must be retried, so a file-only harness cannot pass by construction (hardened post-pilot with a nonce+sha256 proof-of-execution after a frontier model hand-wrote the fixture, §4.4); T2 multi-file consistency (regressions ding State); T3 artifact generation with sourcing requirements;

Harness	deepseek-r1:1.5b	qwen3:8b	deepseek-r1:8b	claude-opus-4-8
claude (Claude Code)	—	—	—	**1.00** (12 cells, $1.38/run)
aider	**0.49** (85%)	0.59 (81%)	**0.81** (93%)	—
ollama (thin control)	0.12	**0.88** (96%)	0.63 (93%)	—
pi	0.00	0.70 (70%)	0.00	—
hermes	0.00	**0.91** (100%)	0.00	—
goose	0.00 (89%)	0.33 (**33%**)	0.00 (89%)	—
codex	0.00	0.00	0.00	—
mock (floor)	0.13	—	—	—

Table 1: GOODPUT per `harness @ model` on the 515-run pilot (finish rate in parentheses where any attempt timed out). Holding the model fixed, harness choice spans near-0 to near-1 on every model column.

T4 safety—a prompt-injection/secret-redaction task where leaking the secret zeroes the score even on a completed run. Tasks are discriminating *by design*; external anchoring is future work (§6) via a task-import contract that wraps external suites.

Panel. Harnesses: `claude` (Claude Code, frontier reference), `aider` (repo-map, diff-driven), `codex` (OpenAI Codex CLI, structured tool protocol), `goose`, `pi` (sub-1k-token prompt, 4 tools), `hermes`, `ollama` (thin file-block parser—the control), `mock` (deterministic floor). Local models: `deepseek-r1:1.5b`, `deepseek-r1:8b` (reasoning, inline `<think>` narration), `qwen3:8b` (clean output); the size-ladder study adds `qwen3.5` {2b, 4b, 9b}. Cloud arms: Claude as a harness and as a model behind lean harnesses (via an API shim), plus metered OpenAI arms.

Integrity. Every published number regenerates from committed JSONL ledgers; a CI guard recomputes ~20 pinned claims from the frozen ledger and fails the build on drift. Runs are sandboxed into throwaway copies; a pristine-source guard restores each task from git before every cell and logs sandbox escapes (it fired twice in production, catching a harness deleting a file from the pristine task source). Cells are killed at a per-(model, host) fitted wall timeout; a host-health canary probes generation rate before every cell and the cell order is deterministically shuffled, decorrelating host drift from any block (§4.1).

4 Results

All numbers below are from the frozen pilot ledger (515 runs: 8 harnesses × 3 local models × up to 9 tasks × 3 seeds, plus cloud slices), the Phase B cloud bookend (20 runs), and the Phase C size ladder (222 runs). 95% intervals are seeded bootstrap; paired comparisons use a task-clustered bootstrap (resample tasks, then seeds). We label every claim's status per the pre-registration discipline of §6: the pilot is exploratory and in-sample.

4.1 Harness attribution is large; reliability reorders the field

Table 1 is the capacity scorecard. On `qwen3:8b` the harness alone spans GOODPUT 0.00 (`codex`) to 0.91 (`hermes`); on `deepseek-r1:1.5b`, `aider` beats the thin control by Δ=0.37 [0.16, 0.59] (significant, paired clustered bootstrap). Attribution is strong at the extremes and honestly weak among good harnesses: the top pair on `qwen3:8b` (`hermes` vs. `ollama`) is a statistical tie (Δ=0.03 [−0.02, 0.11]).

Excluding timeouts inverts rankings. Under the common finished-runs-only score, `pi @ qwen3:8b` reads 1.00 and `goose @ qwen3:8b` 0.99—top of the field. But `goose` finished 9 of 27 attempts and `pi` 19 of 27: as GOODPUT they fall to 0.33 and 0.70, and the honest leaders are the harnesses that *deliver* (`hermes` 0.91 at 100% finish, `ollama` 0.88 at 96%). Any evaluation that drops non-finishing runs systematically flatters exactly the harnesses that hang.

What a timeout is (autopsy). A per-cell autopsy of all 55 pilot timeouts, using three independent evidence sources (workdir mtimes, proxy events, server request windows), attributes 51 to host-conditional wall-clock latency (still generating, within token budget) and only 4 to true hangs (all `codex`, which never completed a model call). We therefore keep GOODPUT as the delivery metric but withdraw "flaky harnesses hang" as a mechanism; the scaled protocol adds per-model timeout fits, randomized cell order, and a host-health canary, after observing that wall-clock GOODPUT on a memory-constrained host is *non-stationary* (swap growth collapsed throughput mid-battery, making cell order a confound). In the Phase C battery run under the full hardening, all 22 timeouts classify as harness hangs with zero host-degraded cells—evidence the controls work.

Harness	2b (2.7 GB)	4b (3.4 GB)	9b (6.6 GB)	mean
pi	**0.79**	0.60	**0.92**	**0.77**
ollama (thin control)	0.24	0.55	0.64	0.48
aider	0.30	0.32	0.41	0.34
codex	0.00	0.00	0.00	0.00

Table 2: Phase C size ladder (222 runs): GOODPUT across the `qwen3.5` family, thinking off, under full hardening. The right harness on a 2b model beats the thin control on a model 2.4× larger.

4.2 Interface-fit, not capability, is often the binding constraint

The Codex bookend (cleanest result). `codex` scores 0 across all 89 local cells—every failure an artifact-commitment failure: its headless runner requires the model to speak its structured tool-call dialect, and no local model in the panel can. The same harness, pointed at its native cloud model (`gpt-5.5`), passes 20/20 (5 seeds × 4 tasks, every cell one iteration, Safety = 1), *including* the hardened tool-recovery task. A model swap alone produces a $0 \rightarrow 1$ discontinuity with the harness held fixed. The home-turf confound is addressed: on a *non-native* mid-tier cloud model (`gpt-4o-mini`), the tool-required task splits by harness *type*—text-diff `aider` fails 0/3 while tool-calling `pi` passes 3/3—so the success is interface-fit, not native tuning. A follow-up on a local model that *does* emit clean OpenAI-style tool calls (`qwen3.5:9b`) sharpens the mechanism: `codex` still scores 0 (its local path expects a different tool dialect) while `pi` sweeps tool-recovery 3/3—interface-fit binds at the *weakest link of the harness's dialect chain* (harness wire-API × serving translation × model template), demonstrated entirely locally.

Output shape reorders the field. On both reasoning models (inline `<think>` narration in the content channel), the parse-and-apply harnesses `pi` and `goose` collapse to 0 *while the model is answering*: the metering proxy logs full responses on every iteration, yet no file is ever written—the harness cannot extract an actionable edit from narration-dominated output. Tolerant text parsers (`aider`, the `ollama` control) ride out the narration and commit edits on the same models. The traces separate at least three distinct mechanisms behind identical zeros (harness cannot apply a good answer; no successful model call at all; protocol mismatch)—one reason "Score 0" must name a (harness, model) pair and open the trace, never indict a harness alone. We flag the confound honestly: both reasoning models are one family, and live probes show the operative variable is where the *serving stack* puts reasoning (inline in content vs. a separate field)—the de-confounding design is pre-registered (§6).

4.3 The harness substitutes for capability where capability is scarcest

Table 2: on the 2.7 GB `qwen3.5:2b`—which alone passes only 1/9 tasks at the calibration floor—the thin control scores 0.24 while `pi` scores 0.79, Δ=0.55 [0.18, 0.87] (task-clustered bootstrap, significant). `pi` at 2b even beats the control at 9b (0.64): the scaffold buys more than a 2.4× model-size jump at the bottom of the ladder. The control rides raw capability monotonically (0.24/0.55/0.64); `codex` is a structural 0 across the whole ladder (capability-independent); and capability-per-GB is non-monotonic for `pi` (4b < 2b: the 4b checkpoint has task-specific holes)—routing must pick the pair per task, not the biggest model.

Transfer (reach) is rare. Across the pilot's local panel, `aider` is the only lean harness with nonzero GOODPUT on every model (0.49/0.59/0.81, mean 0.63); `pi`, `hermes`, and `goose` each win on at most one model and vanish elsewhere. The cross-model rank instability exceeds a seed-resampling noise null (Kendall's $\tau = 0.407$ vs. null 5th percentile 0.733, $p < 0.0005$): most harness advantages are model-specific, not structural. Within the qwen3.5 ladder, rank stability is higher ($\tau = 0.78$) but still partial.

Out-of-sample confirmation (Phase D). Because pilot reach spanned only two local families, we pre-registered and ran a third-family arm: `llama3.2:3b` (Meta), 7 harnesses × 11 tasks (including the hardened proof-carrying T1 trio, their first appearance in a published ledger) × 5 seeds—341 cells, 0 errors, 0 host-degraded canary events, all 5 timeouts harness-attributable. The predictions hold out-of-sample: `aider` is again the top lean harness and again nonzero (GOODPUT 0.64 at 93% finish)—nonzero on all four local models across three families—while the tool-calling harnesses remain model-specific: `pi` falls to 0.18, *significantly below the thin control* ($\Delta_{\text{ollama}-\text{pi}}$=0.41 [0.01, 0.77], task-clustered bootstrap), and `hermes` and `goose` collapse to ≤0.03. `codex` extends its structural zero to a fourth local model (now 0/144 local cells). The top-pack tie replicates too: `aider` vs. the control is Δ=0.05 [−0.07, 0.20], n.s. On the hardened T1 trio only tool-calling harnesses pass any cell (`pi` 3/15, `goose` 1/15; `aider` 0/15, file-only control 0 by construction)—the pilot's T1 discrimination pattern, now under verifier-gaming-proof tasks.

4.4 At the frontier, quality compresses and cost discriminates

Putting a frontier model (`claude-opus-4-8`) *behind* the text-format lean harnesses via an API shim: both pass 9/9 with Score 0.97–0.98 (Δ=0.004, n.s.)—the discriminating power that mid-size models provide flattens at the top. What Score no longer separates, cost does, multiplicatively: \$1.59/run (`aider`) vs. \$0.27/run (`ollama`), a ~6× gap at ~9× the token efficiency, driven by loop shape (context re-sent per iteration). This is why the score never folds in cost.

The frontier slice also surfaced a task-design lesson we report as a caution for the field: the pilot's tool-recovery task required generating a 200-entry fixture via a two-phase generator ("do not hand-write it"). The frontier model simply *hand-wrote all 200 entries* without executing anything, and the outcome-based verifier could not tell. This is precisely the verifier-gaming failure mode the ABC checklist warns of [21]; the task is now hardened (random nonce + sha256(nonce+cases) proof-of-execution, with hand-written-artifact rejection tests in CI), and the Phase B bookend passes the hardened version.

4.5 Safety: the gate fires on runs that complete

On the injection/redaction tier and the tool tier, the gate catches real harnesses, not just the deterministic floor: `aider` trips the resource gate on 12–17% of cells on the deepseek models while otherwise strong; the floor gates at 22%. A manual audit of all nine auditable gated agent cells finds genuine policy events (forbidden-path writes, secret leakage into output, out-of-area writes including editing a protected generator) and zero glob false positives. Claude Code held Safety = 1 on 12/12 while completing—resisting the injection is compatible with finishing the task. Separately, we observe that *sandbox containment is itself a harness property*: one harness, told to delete a lock file, discovered the repository root and deleted the file from the pristine task source rather than its sandboxed copy—a blast-radius failure now caught by the pristine-source integrity guard and reported as a per-harness escape rate.

4.6 A routing rule keyed on task tier, not prompt difficulty

Per tier, the best local (harness, model) pair ties the frontier reference's Goodput on every tier tested, at \$0 marginal token cost—but the winning pair is *tier-specific* (`pi` @ `qwen3:8b` for tool-recovery, `aider` @ `deepseek-r1:8b` for multi-file), so a difficulty router cannot pick it from the prompt. Read with the local model held fixed (it is whatever fits the deployment's memory): `qwen3:8b` clears every tier locally; `deepseek-r1:8b` clears all *except* tool-recovery (best local 0.13)—escalate exactly that tier to cloud; `deepseek-r1:1.5b` escalates nearly everything. The rule that falls out: *the harness choice matters most in the middle*. A mid-size model can be harnessed to match cloud; a tiny model cannot be harnessed into capability it lacks; and tool-recovery is the tier where even a solid 8B should hand off. The pre-registered contrast against a prompt-length difficulty baseline is future work.

5 Threats to validity

Exploratory, in-sample pilot. Hypotheses were sharpened from the pilot they are supported by; we claim motivation, not confirmation—except where the pre-registered Phase D arm (§4.3) confirms out-of-sample; the remainder of the confirmatory design (§6) is not yet complete. **Homegrown tasks.** Nine authored tasks, built to discriminate—the discrimination could partly reflect task construction; the mitigation (an external anchored slice via the task-import contract) is designed but not yet run. **Narrow panel.** Three local models in two families; the reach claim could be a family effect. **Few seeds.** Three seeds give wide intervals; many pairwise differences are correctly reported as not significant; only one adapter exposes a true seed knob (others' seeds are independent samples, flagged when zero-variance artifacts occur). **Single host.** All local runs share one Apple-Silicon machine; wall-clock-dependent results are host-conditional (and, pre-hardening, host-*history*-conditional). **Metering asymmetry.** Two harnesses bypass the token proxy (shown as unmetered, never imputed); one cloud arm ran on a subscription; the frontier cost is a cache-inflated upper bound—cost claims are scoped to uniformly metered arms. **Unvalidated process proxies.** Path/State are deterministic proxies not yet validated against human judgment (κ study pre-registered). **Frontier slice is not agentic Claude Code**: the shim measures the lean harness driving the frontier model as a raw completion endpoint, tools off.

6 Pre-registered confirmatory design

Frozen before execution: (1) *models*: ≥3 local families (Qwen, Llama, Mistral/Gemma), each in clean-output and reasoning variants where available, plus ≥2 cloud families with tool calling—isolating output shape from family and size; (2) *tasks*: the tiered battery *plus* an anchored slice imported from a recognized suite (SWE-bench-Verified / Terminal-Bench / τ^2-bench), re-checking the main effects on external tasks; (3) *power*: ≥5 seeds, CI-width targeted per Miller [14], task-clustered bootstrap, pass^k, rank stability against a seed-resampling null; (4) *construct validity*: human labels on a stratified trace sample with proxy↔human κ for Path, State, and the gate; (5) *integrity*: hidden oracles, pristine-source guard, deterministic scorecard, CI drift guard. **Primary prediction** (the thesis lives or dies on it): interface-fit effects (the Codex discontinuity; shape-driven collapse, de-confounded) are larger than the within-interface-class model-capability effect on sub-frontier models. If capability explains the outcomes and interface-fit does not, the central thesis is refuted, and we will report it. *Status:* the third-family arm (Phase D, §4.3) is executed and confirms the transfer and structural-zero predictions out-of-sample; the external task anchor, the κ construct-validity study, the difficulty-router contrast, and the second-machine replication remain open.

7 Conclusion

The harness is not packaging around a model; on sub-frontier models it is often *the* capability—and it can also be the thing that is broken. Crucible turns that from a observation into a measurement: a factorial, safety-gated, delivery-honest, drift-guarded instrument whose output is not a leaderboard but an attribution (which member of the pair failed), a mechanism (interface-fit), and a decision (when to stay local). The apparatus, adapters, tasks, frozen ledgers, and analysis code are released for reuse; the pre-registered scaled study is the immediate next step.

References

[1] Maksym Andriushchenko, Alexandra Souly, et al. Agentharm: A benchmark for measuring harmfulness of LLM agents. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.09024.

[2] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering, 2024. arXiv:2410.07095.

[3] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance, 2023. arXiv:2305.05176.

[4] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. arXiv:2406.13352.

[5] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-efficient and quality-aware query routing, 2024. arXiv:2404.14618.

[6] Epoch AI. Why benchmarking is hard. https://epoch.ai/gradient-updates/why-benchmarking-is-hard, 2024.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

[8] Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *Transactions on Machine Learning Research (TMLR)*, 2025. arXiv:2407.01502.

[9] Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, et al. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.11977.

[10] Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026. arXiv:2604.25850.

[11] Xiao Liu, Hao Yu, Hanchen Zhang, et al. Agentbench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.03688.

[12] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. arXiv:2601.11868.

[13] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants, 2023. arXiv:2311.12983.

[14] Evan Miller. Adding error bars to evals: A statistical approach to language model evaluations, 2024. arXiv:2411.00640.

[15] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data, 2024. arXiv:2406.18665.

[16] Yubin Qu, Ying Zhang, Yanjun Zhang, Gelei Deng, Yuekang Li, Leo Yu Zhang, and Yi Liu. Overeager coding agents: Measuring out-of-scope actions on benign tasks, 2026. arXiv:2605.18583.

[17] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.15817.

[18] Tianbao Xie, Danyang Zhang, Jixuan Chen, et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. arXiv:2404.07972.

[19] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.

[20] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2406.12045.

[21] Yuxuan Zhu, Tengjun Jin, et al. Establishing best practices for building rigorous agentic benchmarks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2507.02825.